

Frequent Itemset Mining

Aprendizagem Computacional Ciências da Computação

The Problem of Frequent Itemset Mining

- **Frequent Itemset Mining** consists of enumerating all itemsets having a **support** greater or equal to a user-defined value, **minsup**.
- For the user, choosing a **high minsup** value will:
 - ▶ Reduce the number of frequent itemsets found.
 - ▶ Increase processing speed and decrease the memory required.

Frequent Itemset Mining

Transaction Database

$D = \{\{\text{butter, bread, milk, sugar}\}, \{\text{butter, flour, milk, sugar}\}, \{\text{butter, eggs, milk, salt}\}, \{\text{eggs}\}, \{\text{butter, flour, milk, salt, sugar}\}\}$

Tasks

- 1 find all subsets of items that occur together in many transactions.
- 2 find all rules that correlate the presence of one set of items with that of another set of items in the transaction database.

Counts

item	count
butter	4
milk	4
sugar	3
eggs	2
flour	2
salt	2
bread	1

Concepts

- Discovering regularities in transactional data
- Basis for many data mining tasks

Why Frequent Itemsets?

- Large databases contain hidden regularities
- We want groups of items that co-occur often
- Supports decision making (marketing, recommendations, design)

Concepts

Items set of all possible items I

Itemset set of items $X \subseteq I$

Database Set of transactions, each transaction is a set of items

Support $\text{support}(X) = |\{T \in D \mid X \subseteq T\}|$

Frequent itemset X is frequent iff $\text{support}(X) \geq \text{minsup}$

Apriori algorithm

The apriori principle (anti-monotonicity)

- Any non-empty subset of a frequent itemset is frequent
- Any superset of a non-frequent itemset is non-frequent

Method

- Start by building all frequent 1-itemsets
- Only consider $(k+1)$ -itemsets when all subsets of length k are also frequent

Apriori algorithm

Concepts

- Use lexicographic order for all items
- Joining two k -itemsets only if they share the first $k-1$ items

Variables

C_k candidate itemsets of size k

L_k frequent itemsets of size k

Produce candidates

Join Join L_k with itself to produce C_{k+1}

Prune Discard itemsets from C_{k+1} that contain non-frequent k itemsets

Prove candidates

- Count all candidates from C_{k+1} using the database
- Only keep those with minsupport to make L_{k+1}

Apriori example

$D = \{\{1,3,4,6\}, \{2,3,5\}, \{1,2,3,5\}, \{1,5,6\}\}$ minsup = 2

C_1

item	count
1	3
2	2
3	3
4	1
5	3
6	2

L_1

item	count
1	3
2	2
3	3
5	3
6	2

Apriori example

$D = \{\{1,3,4,6\}, \{2,3,5\}, \{1,2,3,5\}, \{1,5,6\}\}$ minsup = 2

C_2

item	count
$\{1,2\}$	1
$\{1,3\}$	2
$\{1,5\}$	2
$\{1,6\}$	2
$\{2,3\}$	2
$\{2,5\}$	2
$\{2,6\}$	0
$\{3,5\}$	2
$\{3,6\}$	1
$\{5,6\}$	1

L_2

item	count
$\{1,3\}$	2
$\{1,5\}$	2
$\{1,6\}$	2
$\{2,3\}$	2
$\{2,5\}$	2
$\{3,5\}$	2

Apriori example

$D = \{\{1,3,4,6\}, \{2,3,5\}, \{1,2,3,5\}, \{1,5,6\}\}$ minsup = 2

C_3

$C_3 =$
 $\{\{1,3,5\}, \{1,3,6\}, \{1,5,6\}, \{2,3,5\}\}$

Pruning C_3

$\{1,3,6\}$ because $\{3,6\} \notin L_2$
 $\{1,5,6\}$ because $\{5,6\} \notin L_2$

C_3

item	count
$\{1,3,5\}$	1
$\{2,3,5\}$	2

L_3

item	count
$\{2,3,5\}$	2

Apriori example

$D = \{\{1,3,4,6\}, \{2,3,5\}, \{1,2,3,5\}, \{1,5,6\}\}$ minsup = 2

item	count
{1}	3
{2}	2
{3}	3
{5}	3
{6}	2
{1,3}	2
{1,5}	2
{1,6}	2
{2,3}	2
{2,5}	2
{3,5}	2
{2,3,5}	2

Apriori Algorithm

- Uses frequent k -itemsets to generate candidate $k + 1$ -itemsets
- Needs to scan database to get counts of frequent itemsets
- Needs a large number of candidate sets
- Needs to scan the database a large number of times

Association rules

$$\text{support}(A \implies B) = \text{support}(A \cup B)$$

$$\text{confidence}(A \implies B) = \text{support}(A \cup B) / \text{support}(A)$$

Association rules

Counts

item	count
{1}	5
{3}	3
{5}	4
{1,3}	3
{1,5}	4
{3,5}	2
{1,3,5}	2

Generating Association rules from Counts

rule	support	confidence
$\{1\} \Rightarrow \{3\}$	3	3/5
$\{1\} \Rightarrow \{5\}$	4	4/5
$\{3\} \Rightarrow \{1\}$	3	3/3
$\{3\} \Rightarrow \{5\}$	2	2/3
$\{5\} \Rightarrow \{1\}$	4	4/4
$\{5\} \Rightarrow \{3\}$	2	2/4
$\{1\} \Rightarrow \{3, 5\}$	2	2/5
$\{3\} \Rightarrow \{1, 5\}$	2	2/3
$\{5\} \Rightarrow \{1, 3\}$	2	2/4
$\{1, 3\} \Rightarrow \{5\}$	2	2/3
$\{1, 5\} \Rightarrow \{3\}$	2	2/4
$\{3, 5\} \Rightarrow \{1\}$	2	2/2

Frequent Pattern Trees

- Divide and conquer methodology
- Avoids candidate generation
- Highly condensed data structure
- Only needs to scan the database twice:
 - 1 To count item frequencies
 - 2 To build the FP-tree

Construct FP-tree using $\text{min-sup} = 3$

Database	Ordered Frequent Items
{f, a, c, d, g, i, m, p}	{f, c, a, m, p}
{a, b, c, f, l, m, o}	{f, c, a, b, m}
{b, f, h, j, o}	{f, b}
{b, c, k, s, p}	{c, b, p}
{a, f, c, e, l, p, m, n}	{f, c, a, m, p}

item	count
f	4
c	4
a	3
b	3
m	3
p	3

Building FP-Tree

Ordered Frequent
Items

{f, c, a, m, p}

{
|
f:1
|
c:1
|
a:1
|
m:1
|
p:1

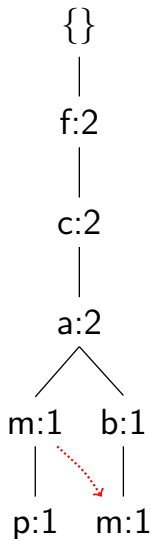
Building FP-Tree

Ordered Frequent Items

{f, c, a, m, p}

{f, c, a, b, m}

Red pointers are used to facilitate the generation of frequent terms



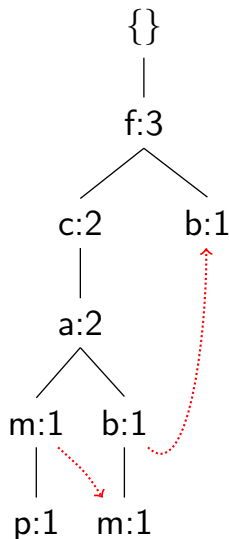
Building FP-Tree

Ordered Frequent Items

{f, c, a, m, p}

{f, c, a, b, m}

{f, b}



Building FP-Tree

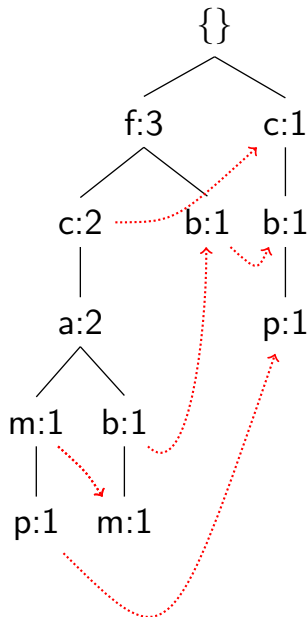
Ordered Frequent Items

{f, c, a, m, p}

{f, c, a, b, m}

{f, b}

{c, b, p}



Building FP-Tree

Ordered Frequent Items

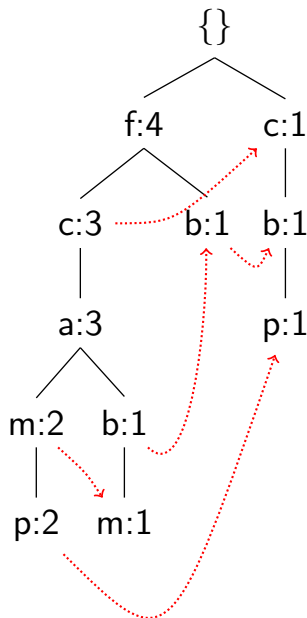
{f, c, a, m, p}

{f, c, a, b, m}

{f, b}

{c, b, p}

{f, c, a, m, p}



Benefits of FP-Tree

- Infrequent items are gone
- Doesn't break a long pattern
- Preserves complete information
- Frequent items are likely shared

FP-Growth: Frequent Terms

- All frequent terms are obtained from the FP-tree
- We follow the links from the header table
- Start by the least frequent items

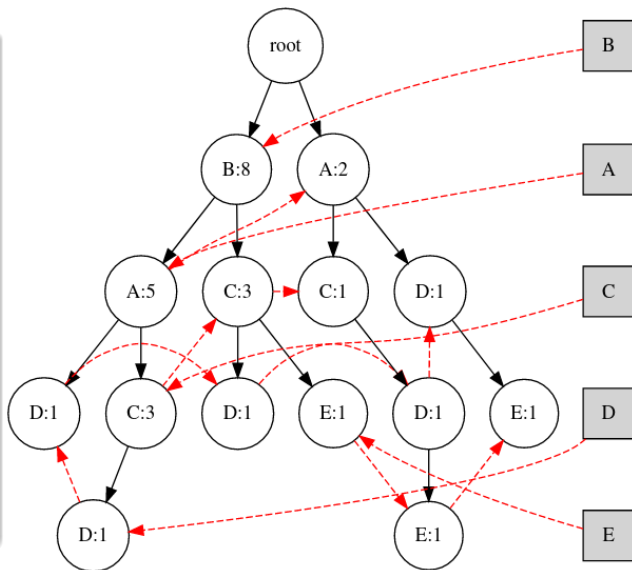
Algorithm

- 1 For each item l (from least frequent to most frequent)
- 2 Get all itemsets that contain l (conditional pattern base)
- 3 Get the conditional FP-tree from these itemsets
- 4 Get itemsets of size 2 from the conditional FP-tree
- 5 Repeat step 2 for each itemset of size 2

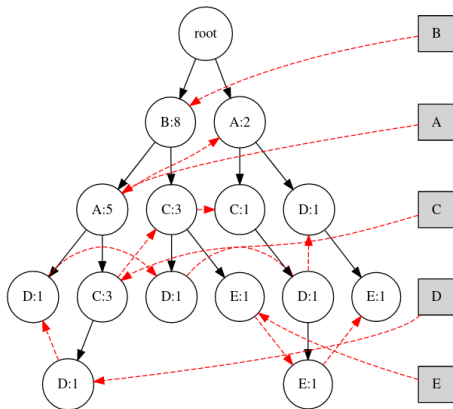
Building FP-Tree minsup = 3

Transactions

Before	After
ABFI	BA
BCD	BCD
ACDE	ACDE
ADE	ADE
ABC	BAC
ABCD	BAC
BCG	BC
ABCH	BAC
ABD	BAD
BCEH	BCE

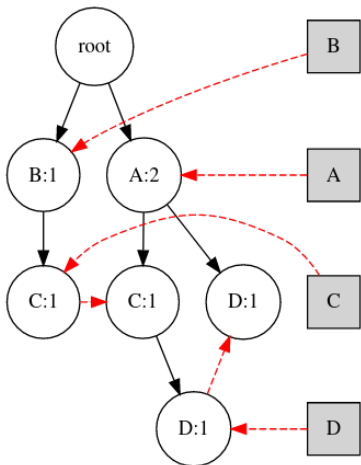


First step



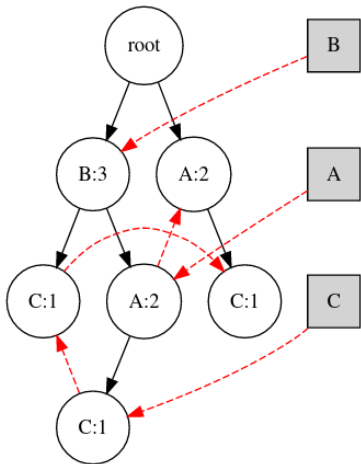
Item	Pattern base
E	$\{\{B:1,C:1,E:1\}, \{A:1,C:1,D:1,E:1\}, \{A:1,D:1,E:1\}\}$
D	$\{\{B:1,A:1,C:1,D:1\}, \{B:1,A:1,D:1\}, \{B:1,C:1,D:1\}, \{A:1,C:1,D:1\}, \{A:1,D:1\}\}$
C	$\{\{B:3,A:3,C:3\}, \{B:3,C:3\}, \{A:1,C:1\}\}$
A	$\{\{B:5,A:5\}, \{A:2\}\}$
B	$\{\{B:8\}\}$

Fp-tree for E

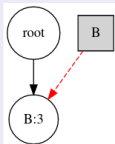


No tree for minsup = 3

Fp-tree for D



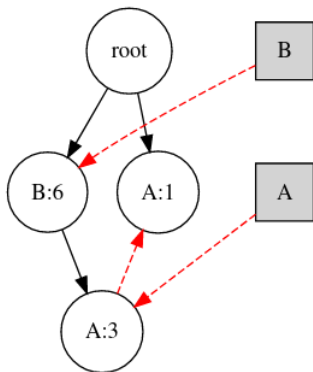
Tree with minsup = 3



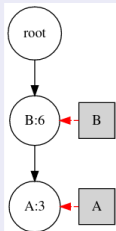
Sets

{B, D}

Fp-tree for C



Tree with minsup = 3



Sets

{A,C}

{B,C}

{B,A,C} recursively

Frequent sets found by FP-Growth

$\{B\}, \{A\}, \{C\}, \{D\}, \{E\}$

$\{B, D\}, \{A, C\}, \{B, C\}, \{B, A\}$

$\{B, A, C\}$

The ECLAT Algorithm

- **ECLAT** stands for **Equivalence CLAss Transformation**.
- Developed by **Mohammed Javeed Zaki** in 2000.
- Key characteristics:
 - ▶ Generally **faster than Apriori**.
 - ▶ Utilizes a **depth-first search**.
 - ▶ Uses a **vertical database** representation.
 - ▶ Uses **equivalence classes** of itemsets sharing the same prefix.

Equivalence Classes

- Let there be two itemsets X and Y of size k .
- X and Y belong to the same equivalence class if the $k - 1$ first items of X and Y are the same according to the total order.
- e.g. An equivalence class:

{pasta, lemon, bread},
{pasta, lemon, orange},
{pasta, lemon, cake}

Visualising the Search Space

- The search space can be visualised as a **Hasse diagram** or a **set enumeration tree**.
- Nodes represent itemsets, and edges represent the addition of items according to the total order.
- ECLAT explores this space using a depth-first approach.

The Transaction Database

Consider the following database with **minsup** = 2:

TID	Items
T1	{pasta, lemon, bread, orange}
T2	{pasta, lemon}
T3	{pasta, orange, cake}
T4	{pasta, lemon, orange, cake}

- **Total Order:** pasta $\prec$ lemon $\prec$ bread $\prec$ orange $\prec$ cake.

Step 1: Create Vertical Representation

ECLAT scans the database once to create **TID-lists** for each item:

- **pasta**: {T1, T2, T3, T4} (Support: 4)
- **lemon**: {T1, T2, T4} (Support: 3)
- **bread**: {T1} (Support: 1)
- **orange**: {T1, T3, T4} (Support: 3)
- **cake**: {T3, T4} (Support: 2)

Step 2: Initial Filtering

- The algorithm identifies the first equivalence class and eliminates **infrequent** itemsets.
- **{bread}** has a support of 1, which is less than **minsup = 2**, so it is **eliminated**.
- **Frequent itemsets (size 1):** {pasta}, {lemon}, {orange}, {cake}.

Step 3: Candidate Generation (Size $k + 1$)

Combine size-1 itemsets to generate size-2 candidates using **TID-list intersections**:

- **{pasta, lemon}**: $\{T1, T2, T3, T4\} \cap \{T1, T2, T4\} = \{\mathbf{T1}, \mathbf{T2}, \mathbf{T4}\}$ (Sup: 3)
- **{pasta, orange}**: $\{T1, T2, T3, T4\} \cap \{T1, T3, T4\} = \{\mathbf{T1}, \mathbf{T3}, \mathbf{T4}\}$ (Sup: 3)
- **{pasta, cake}**: $\{T1, T2, T3, T4\} \cap \{T3, T4\} = \{\mathbf{T3}, \mathbf{T4}\}$ (Sup: 2)
- **{lemon, orange}**: $\{T1, T2, T4\} \cap \{T1, T3, T4\} = \{\mathbf{T1}, \mathbf{T4}\}$ (Sup: 2)
- **{orange, cake}**: $\{T1, T3, T4\} \cap \{T3, T4\} = \{\mathbf{T3}, \mathbf{T4}\}$ (Sup: 2)

Step 4: Recursive Depth-First Processing

Process new equivalence classes (e.g., those starting with “pasta”) to find size-3 itemsets:

- **{pasta, lemon, orange}**: $\{T1, T2, T4\} \cap \{T1, T3, T4\} = \{T1, T4\}$ (Sup: 2)
- **{pasta, orange, cake}**: $\{T1, T3, T4\} \cap \{T3, T4\} = \{T3, T4\}$ (Sup: 2)
- **{pasta, lemon, cake}**: $\{T1, T2, T4\} \cap \{T3, T4\} = \{T4\}$ (Sup: 1)
→ **Eliminated.**

Final Results of the Example

The frequent itemsets discovered are:

- **Size 1:** {pasta}, {lemon}, {orange}, {cake}
- **Size 2:** {pasta, lemon}, {pasta, orange}, {pasta, cake}, {lemon, orange}, {orange, cake}
- **Size 3:** {pasta, lemon, orange}, {pasta, orange, cake}

Total itemsets explored: 14 (compared to 18 for Apriori).

Performance Overview

- ECLAT requires only a **single database scan**.
- The most costly operation is the **intersection of TID-lists**.
- Performance is significantly affected by the **total order** chosen for items.

Key Optimizations

- **Optimization 1 (Total Order):** Sorting items by **increasing support** is more efficient than alphabetical order, as items with lower support produce smaller sub-trees.
- **Optimization 2 (Intersections):** Using **bit vectors** to represent TID-lists speeds up logical intersections.
- **Optimization 3 (Memory):** Equivalence classes sharing a prefix can be stored more efficiently by separating the prefix from the unique suffixes.

Bit Vectors

TID

- **pasta:** {T1, T2, T3, T4} (Support: 4)
- **lemon:** {T1, T2, T4} (Support: 3)
- **bread:** {T1} (Support: 1)
- **orange:** {T1, T3, T4} (Support: 3)
- **cake:** {T3, T4} (Support: 2)

Bit Vectors

- **pasta:** 1111
- **lemon:** 1101
- **bread:** 1000
- **orange:** 1011
- **cake:** 0011

Bit Vectors

Bit Vectors

- **pasta**: 1111
- **lemon**: 1101
- **bread**: 1000
- **orange**: 1011
- **cake**: 0011

Intersection

- **{pasta, lemon}**: $1111 \& 1101 = \mathbf{1101}$ (Sup: 3)
- **{pasta, orange}**: $1111 \& 1011 = \mathbf{1011}$ (Sup: 3)
- **{pasta, cake}**: $1111 \& 0011 = \mathbf{0011}$ (Sup: 2)
- **{lemon, orange}**: $1101 \& 1011 = \mathbf{1001}$ (Sup: 2)
- **{orange, cake}**: $1011 \& 0011 = \mathbf{0011}$ (Sup: 2)

Pseudocode of ECLAT

The following algorithm describes the recursive process of ECLAT using prefix-based equivalence classes:

```
ECLAT(an equivalence class C)
FOR EACH X in C:
    T = empty_set
    FOR EACH Y in C such that X < Y:
        R = X union Y
        t(R) = t(X) intersection t(Y)
        IF support(R) >= minsup:
            Output R
            T = T union R
    IF T is not empty:
        ECLAT(T)
```

Market Basket Analysis

- Discover sets of products frequently bought together
- Examples:
 - ▶ {bread, butter}
 - ▶ {diapers, beer}
- Used for:
 - ▶ Shelf organization
 - ▶ Cross-selling and promotions
 - ▶ Product bundling

Recommendation Systems

- Use frequent itemsets as candidate recommendations:
 - ▶ “Customers who bought (X) also often bought (Y)”
- Can be combined with:
 - ▶ Collaborative filtering
 - ▶ Content-based methods
- Helps generate simple, interpretable suggestions

Web Usage Mining

- Log data: each session = set or sequence of pages
- Frequent itemsets reveal:
 - ▶ Pages often visited together
 - ▶ Common navigation patterns
- Use cases:
 - ▶ Website redesign
 - ▶ Pre-fetching and caching
 - ▶ Targeted content

- Transactions: genes, proteins, SNPs, or motifs
- Frequent itemsets capture:
 - ▶ Co-expressed genes
 - ▶ Frequently co-occurring mutations
- Potential uses:
 - ▶ Hypothesis generation in biology
 - ▶ Identifying candidate biomarkers

Text and Log Mining

- Text:
 - ▶ Treat documents as transactions
 - ▶ Items are words or phrases
 - ▶ Frequent itemsets correspond to topics or key term groups
- Logs:
 - ▶ Items are events or error codes
 - ▶ Identify frequent co-occurring failures for root-cause analysis